HeavyHeader

C/C++ 构建效能治理平台

把「删掉这行 include 会怎样」变成一次可复现的受控实验

第四届开放原子大赛 · 2026 开源行业解决方案创新赛
赛道一 · 开源基础软件与解决方案

项目负责人：王子周
申报主体类型：开源社区（个人）
日期：2026 年 10 月 19 日

代码仓库：`https://github.com/wzzrambler/heavyheader`

Apache-2.0 · 本作品以开源方式提交

摘要

大型 C/C++ 工程的构建时间是真实的成本：一次全量构建动辄数十分钟到数小时，CI 集群机时与工程师等待时间都花在里面。构建慢的最大单一来源往往不是算法，而是头文件失控——一个被数万个编译单元传递包含的重量级头文件，会让整个项目反复为它付费。

现有工具分别解决了这个问题的一段，本文先把来源交代清楚：Header Hero（Bitsquid，2011）用「头文件行数 × 被包含次数」给出成本排序，本工具的归因行数与 Blowup Factor 公式即来自它；ClangBuildAnalyzer 用 `-ftime-trace` 给出逐头文件的累计毫秒与被包含次数，CompileScore 把同一数据源做成了 VS 内的可视化（每 TU 时间线、includers 图）；IncludeGuardian 给出七类结构建议；deheader（ESR，2010）注释掉一条 include 再编译来验证可删性。但它们都不回答「删掉这一条，这次构建具体省多少时间、多少内存、多少体积，这个差异是不是噪声」——要么没有反事实，要么只有二值判断。HeavyHeader 补的正是这一段，并把它命名为反事实构建实验（Counterfactual Build Experiment，CBE）：让真实编译器报告自己做了什么，对候选做交替 A/B 真实编译，用越过噪声线的实测差值给出「建议删除 / 保留 / 需人工判断」三态结论。

「组合现成工具」与「方法论创新」的界线是可以消融检验的：拿掉本项目中任何一层，都有实测记录在案的错误结论在等着——拿掉独占闭包计算，候选排序会把「所有单元都必需的头」排第一（实测前 5 个候选全部编译失败）；拿掉交替测量，2.005% 的机器漂移会被整块报成优化收益；拿掉强制复测，第一轮 4 条 improved 复测 0/4 复现；拿掉三态决策层，用户手里会剩下 187 条「疑似未使用」的假阳性。完整消融表见仓库 `docs/innovation.md`。

项目已实现九个命令：`scan`（成本归因与报告）、`verify`（实测验证）、`gate`（CI 门禁）、`compare`（前后对比）、`blame`（成本考古：把回归归因到具体 commit 与 include 行）、`edge-cost`（每条 include 边的边际解析成本）、`toolchains`（多编译器横评）、`history`（成本趋势看板）、`pr-comment`（PR 评论机器人），支持 GCC 9+、Clang 17+ 与 MSVC 三种工具链；544 个单元测试，CI 三个工作流共 16 个岗位（3 系统 × 4 个 Python 版本的 12 组测试矩阵 + 代码检查 + 自举扫描 + 覆盖率 + MSVC 证据），当前全部通过；运行时零第三方依赖，且由测试守护（新增依赖会让 CI 直接失败）。

除成本归因外，项目还做了四件现有工具不做的事：模板实例化成本归因（实测包含 nlohmann/json.hpp 的文件有 39% 构建时间花在模板实例化上）、预编译头收益估算、前向声明建议，以及成本考古——把成本增量归因到具体 commit 与 include 行，2026-10-19 全网搜查未见同能力实现。

在 4 个真实开源项目（fmt / nlohmann-json / spdlog / cpp-httplib）上共扫描 54 个编译单元、1430 万行头文件解析量，产出 87 个成本候选。候选清单包含每一条直接 include，系统头与第三方头也在内——被删的那一行永远写在我们自己的源文件里，删掉 <iostream> 是完全正常的改动；早先版本把系统头整体排除在候选之外，结果恰好把对照实验里 3/3 个有效改动全部挡在门外。头文件来源（项目 / 系统 / 第三方）在报告里只作为标签，不是过滤器。在刻意构造的基准工程上，它准确识别出唯一一条未使用的 include 并实测节省 200 ms（38.5%）。

关键词：构建效能；头文件成本归因；include 治理；开发者工具；CI 门禁；编译器诊断输出

评审关注点导览

评审项（权重）	本文对应章节	一句话答案
技术创新（30%）	第二章（范式与消融表）、第三章 3.5（四道门槛）、1.2.1（同任务对照）、6.4–6.5 与 8.3	反事实构建实验 + 两项新增能力（成本考古 blame、边级归因 edge-cost）；每层创新都有消融证据，跑输的那部分在 8.3 如实写出来
场景落地（30%）	第三章（完整任务示例）、第四章（真实工程 + 完整构建对照 + 上游 PR）	scan → verify → gate 接成流水线；63 TU 完整构建 −11.4% CPU、断言逐个一致；两条实测优化已做成上游 PR 补丁
开源治理（20%）	第九章（治理结构、CI 16 岗位、测试策略）、第十章（缺陷与路线图）	GOVERNANCE + 零依赖守护测试 + 语义化版本发布；CI 三系统 × 四 Python 版本矩阵当前全绿
长期发展（20%）	第十章（Roadmap 与团队）、第九章 9.1（CI 与自举）	v0.1–v0.4 已交付（v0.4 = 成本考古 + 边级归因）、v0.5 生态集成计划中；贡献者阶梯与基金会孵化路径写在 GOVERNANCE.md

一、问题背景与需求依据

1.1 构建慢的钱花在哪

一个 C++ 项目的预处理与解析成本，一阶近似正比于「编译器被迫读了多少头文件文本」。每个被解析的头文件，在每个引用它的编译单元里贡献一次自己的行数。这意味着一个 4700 行的头文件被 3000 个编译单元引用时，项目要为它解析 1410 万行——而这还只是它自身，不含它传递拉进来的一切。

这个成本是隐形的。开发者看到的是「编译好慢」，看不到是哪几行 `#include` 造成的。现有的分析工具要么只做结构展示，要么只做正确性判断，没有一个把成本和具体代码行对应起来。

1.2 竞争格局：每个工具答到哪一句为止

下表是本项目 2026 年 10 月对同类工具的逐项核读（事实核实日期见 `docs/competitors.md`，逐格证据等级见 `docs/competitor-matrix.md`）。对比的重点不是功能有无，而是每个工具答到哪一句为止——「谁贵」「现在花了多少」「该改什么结构」「还能编吗」「该不该包含」都已有成熟答案；空着的是下一句。

工具（出处年份）	回答什么问题	停在哪一句（答不了什么）
Header Hero（2011）	谁可能贵：归因行数排序 + Blowup Factor	只排序；正则解析漏宏与条件编译；不验证、不量化删后收益

工具（出处年份）	回答什么问题	停在哪一句（答不了什么）
ClangBuildAnalyzer	现在谁花了多少：逐头文件累计 ms（真实 -ftime-trace）	只描述现状，没有「删了会怎样」；仅 Clang；无基线、无门禁
CompileScore	同上 + 可视化：每 TU 时间线、includers 图、VS 内着色	同上；无反事实、无决策、无门禁；Windows / VS 为主，无行数归因
MSVC C++ Build Insights	MSVC 侧的官方答案（含 PCH 该放什么）	同上；删后量化、三态决策、按实测净收益的门禁均不提供
IncludeGuardian	该改什么结构：七类建议（guard、PCH、拆分…）	不编译、不实测（token 口径估算）；无三态；仅 Win/Linux 二进制
deheader（2010）	删了还能编吗：注释掉一条 include 再编译	二值结论，不量化收益；整段跳过 #if 区域；无收益阈值
IWYU / clang-include-cleaner	该不该包含：AST 级必需性判定	不给成本量化；同工程同任务实测 0 条删除建议（见 1.2.1）
Include-Wrangler	每条 include 的边际：省掉它少打开几个文件	只数文件、不测时间（2012 年，已休眠）；自写扫描器，不碰编译器
clang-build-profiler	include 树逐边解析耗时 + 模板实例化树	纯呈现：不算「删了省多少」；无验证、无决策、无门禁
include-what-costs	逐头文件实测编译耗时与内存占用	节点级而非逐条语句（LHCb 出品）；无三态结论，无基线门禁
HeavyHeader（本项目）	删了的净收益是多少、有多大把握：反事实实验 + 三态结论 + 门禁	候选池精度不如 AST 级（222 个候选里 187 个编译不过，如实报）

需要说明的是：上表对本项目一栏的描述不是设计目标而是实测结论，其兑现程度由第三、四章的实验数据检验——包括两条不利数据（四工程可采纳率 1/16、1/16、0/16；与 ClangBuildAnalyzer 的排序一致性只有 ρ ≈ 0.68）。「谁贵」这条赛道上已有至少四种成熟答案且都来自编译器内省，因此本项目不把排行榜当卖点（见第二章的范式定义）。

1.2.1 同任务对照：说我们更强之前，先跑一个真竞品

「与现有工具相比优势是否成立」这个问题，只有跑一个真竞品才算回答。项目在同一工程、同一编译器、同一任务上做了两组对照：

对 clang-include-cleaner（LLVM AST 级，业内标准做法）：nlohmann/json 64 个编译单元上，它给出 0 条删除建议、101 条补充建议（它管正确性，我们管成本）；我们把 222 个候选逐条真实编译，得到 1 条实测 −43.5% 的删除建议。两条必须一起读的诚实事实：它分析失败的那一个单元恰是我们实测到 −41.4% 的那个——它的「0 条删除」部分源于没看到这个单元；而它给出的 101 条补充建议我们一条也给不出。结论是互补而非胜出。

对 ClangBuildAnalyzer：同一批 25 个编译单元、同一编译器、同一候选筛选规则，把两边口径统一成「自身耗时 × 次数」后，Top-10 重合 7/10、Spearman ρ ≈ 0.68（两个独立工程上分别是 0.68

/ 0.57）。这组数字支持「中度一致」，不支持「我们更准」——行数归因与真实解析耗时只是中度相关，这正是本工具坚持「行数只用来筛候选、成本结论只能来自实测」的原因。

1.3 目标场景

本项目锚定的场景是有持续集成的 C/C++ 工程的头文件成本治理——团队的代码库已经慢到值得治理，构建由 CI 反复执行，任何结论都必须可复现、可门禁。这个场景对工程规模不设下限：15 个编译单元的 zlib 与 63 个编译单元的 nlohmann/json 测试目标都在本文的实测范围内。

场景的上限是汽车电子、工业软件、通信设备厂商的 C++ 单体仓库（monorepo）：代码量数十万到数百万行、编译单元数以千计、一次全量构建数小时、CI 集群机时是可观成本项。指标设计（归因行数、独占闭包、增量扫描）均以那个规模为前提——归因行数是确定性的，因此能作为基线与门禁指标。

验证场景与目标场景的差距如实说明：本文第三、四章的全部实测来自开源工程（nlohmann/json、doctest、zlib、fmt、spdlog、cpp-httplib，单个工程 15–63 个编译单元）与开源工程的上游贡献（contrib/upstream/）；工业规模单体仓库上的验证尚未进行，这是承认的边界而非隐藏的假设——门禁指标在大规模下的行为由构造样本与真实小工程双重检验，但「3000 个编译单元的 monorepo 上的工程开销」只有 25 TU 的实测值（1.15×–1.20×）可以依据。

二、核心方法论：不自己解析 C++

整个项目只有一条设计原则，其他一切都是它的推论：不自己解析 C++。工具向真实编译器提问，读取它的回答。

这不是懒，是正确性要求。一个手写预处理器在遇到模板元编程、条件编译、宏拼接时会静默给出错误答案，而这种错误比崩溃危险得多——崩溃会被发现，错误答案不会。让编译器自己报告它做了什么，分析结果就在构造上是精确的。

2.0 范式：反事实构建实验

本项目把自己做的事命名为反事实构建实验（Counterfactual Build Experiment，CBE）：工具的输出单位不是一份排序、一张清单或一条 lint 违规，而是一次受控实验的结论。竞品回答「谁贵」（Header Hero、ClangBuildAnalyzer）、「缺什么」（IncludeGuardian）、「还能编吗」（deheader）、「该不该包含」（IWYU、clang-include-cleaner）；HeavyHeader 回答的是一个反事实问题：「如果这行不存在，构建到底会快多少，这个差异是不是噪声。」

一次 CBE 有五个部件，缺任何一件，结论都会错（每件的错误形态都有实测记录，见 2.5 消融表）：

部件	做什么	为什么不能少
① 反事实构造	注释掉候选 include、其余一切不变，真实重编（只写临时目录，原始文件不动）	模拟替代不了编译：宏、条件编译、模板都会让模型偏离

部件	做什么	为什么不能少
② 边际归因选候选	独占闭包算出「删掉这条边的净收益」，不是节点的绝对成本	还能从别的 include 到达的头，删了不省钱（2.3）
③ 统计判定	会话内噪声线 + 每侧 min-of-3 + B/P/B/P 交替测量 + improved 强制复测	单遍读数会把机器漂移当收益：实测单遍 3.738% 复测为 −0.548%
④ 三态可审计结论	建议删除 / 保留 / 需人工判断，每条附三维度证据与复现命令	「疑似」是把判断成本转嫁给用户
⑤ 公开协议与负面结果	预注册门槛、A/A 空对照、三工程 1/16·1/16·0/16 全部公开	一个不公开证伪条件的方法不是方法，是宣传

2.1 一次编译，三个问题

问题	怎么问编译器
你包含了什么，嵌套多深?	GCC/Clang `-H`，MSVC `/showIncludes`
时间花在哪?	GCC/Clang `-ftime-report`，MSVC 墙钟计时
哪些头文件缺 include guard?	`-H` 输出的 trailer

三个问题在同一次编译里全部得到回答——`-H` 和 `-ftime-report` 都写 stderr。分成两次调用会让扫描耗时翻倍，更糟的是两次之间头文件若发生变化，两边数据会互相矛盾。

2.2 归因行数：成本的度量

```
attributed_lines(h) = line_count(h) × tu_reach(h)

tu_reach(h) = 至少间接包含过 h 的编译单元数量
```

这个乘积衡量「整个项目为 h 付出了多少行解析量」。它是对同一份代码确定的，不随机器负载波动——这一点很重要，因为它是 CI 门禁唯一可依据的指标。

2.3 独占闭包：收益的计算

删掉一条 include 后，唯一会停止被解析的头文件集合叫它的独占闭包：

```
exclusive(d) = closure(d) − ( ∪ closure(d′) )
d′ ∈ direct_includes(t), d′ ≠ d
```

这是本项目与普通 lint 工具的本质区别。一个头文件如果能从别的 include 到达，那么删掉当前这条 include 并不会让它停止被解析——钱已经花了。只有独占闭包里的头文件才是删掉这条 include 的真实收益。不做这个计算的工具，会把大量零收益的建议排在真收益前面。

2.4 重复解析检测：确定性的必赚项

`-H` 只为实际处理过的包含打印一行。带 `#pragma once` 或 guard 宏的头文件，第二次及以后的包含会被跳过，因此无论多少个头文件包含它，`-H` 里只出现一次。于是「同一编译单元内出现多次」在物理上只可能是缺少 guard。

修复（补 guard）永远安全，且收益立竿见影。这是报告里唯一标为「必赚项」的分类——不需要实测，因为逻辑上不可能出错。

2.5 消融表：为什么这不是「堆叠现成工具」

评判「组合」与「创新」有一条可操作的标准：拿掉组合中的任一层，结论会不会错。本项目对每一层都做过消融视角的检验，且每条都有仓库内可复现的实测记录：

拿掉哪一层	会得到的错误结论	记录在哪
独占闭包判定	前 5 个候选全部编译失败；真实收益 6.8% 的改动被排到第 25 位	experiments/rank_check.py
交替测量协议	2.005% 的机器漂移被报成优化收益（交替后 0.644%）	experiments/g2-interleaved-aa-*
强制复测	第一轮 4 条 improved 复测 0/4 复现；3.738% 单遍收益是噪声	experiments/g2-confirm-picks-*
三态决策层	用户手里剩下 187 条「疑似未使用」的假阳性	experiments/decision_check.py
白卷诊断	0 条候选被误读成「构建没问题」，而真相是逐条判过、都删不动	experiments/g2_diagnosis.py
门禁最小 TU 保护	构建损坏时成本「下降 67.7%」→ 门禁放行	experiments/g3_real.py

堆叠不做噪声统计、不复测、不公开负面结果。这张表是「方法论创新」与「把工具串起来」之间最直接的区分。

三、完整任务示例（可手算复核）

本章用一个 45 行的构造工程走完「扫描 → 定位 → 验证」全流程。所有数字都可手工复核，工程在仓库 `benchmarks/cases/demo`。

3.1 工程构造

```
benchmarks/cases/demo/
├── include/
│   ├── core.h # 被两个 TU 引用（成本热点）
│   ├── heavy.h # 传递闭包很大
│   ├── bloated.h # 只被 lonely.h 到达的重量级闭包
│   ├── lonely.h # 被 main.cpp 包含但从未使用
│   ├── unguarded.h # 刻意不加 guard
│   └── mid.h
└── app/
    ├── main.cpp # 包含 lonely.h，但代码里不用它
    └── util.cpp
```

3.2 扫描结果

```
$ heavyheader scan benchmarks/cases/demo \
--compiler g++ --std c++17 -I benchmarks/cases/demo/include

2 TU · 425 条依赖边 · 206,106 行头文件解析量 · 5 个成本候选（其中 1 个疑似未使用）
```

成本热点表把 `core.h` 排在首位（被两个编译单元引用），而 `lonely.h` 被标记为「疑似未使用」并因独占闭包最大（22,312 行）排在候选首位。缺 guard 的 `unguarded.h` 出现在「必赚项」表中：解析 3 次、涉及 2 个 TU。

3.3 实测验证

成本模型再严谨也仍是模型。`verify` 命令注释掉候选 include、重新编译、计时对比，每侧重复 3 次取最小值（噪声只会让测量变慢，不会变快，所以最小值是最接近真实成本的估计）：

```
$ heavyheader verify benchmarks/cases/demo --top 3 --runs 3

## lonely.h ← main.cpp
状态: improved — 基线 520 ms → 移除后 320 ms
实测节省: 200 ms (38.5%)

## core.h ← util.cpp
状态: compile-error — 移除后编译失败:
util.cpp:3:14: error: 'Core' was not declared in this scope

## core.h ← main.cpp
状态: compile-error — 移除后编译失败:
main.cpp:6:14: error: 'Core' was not declared in this scope

小结: 1/3 个候选实测有效。
```

注意最后那个「1/3」。工具没有把三个候选都说成可删——它把两个正在使用的头文件正确判为「删了会编译失败」，只对真正未使用的那个给出收益数字。这个区分是第四章在成熟项目上验证的重点。

3.4 明确结论：每个候选一个「值不值得删」

「疑似未使用」是可能性，不是判断。决策层把探针证据收敛成三个明确结论之一：建议删除（编译通过且耗时下降超过 2% 噪声线）、保留（编译器已否决，不必再试）、需人工判断（编译可行但收益在噪声内，附倾向与理由）。未实测的候选显示「未实测」——没测过就不下判断。

每个结论按三个维度列依据：功能影响（还能不能编、哪些标识符消失）、维护成本（整个项目少解析多少代码、还有几个使用单元）、构建体验（编译时间、编译器内存分配、目标文件体积各差多少）。工具链不提供的维度写「该工具链不提供」，不写成 0；报告附完整复现命令。

在 nlohmann/json 测试目标上（222 个候选全部真实编译）：建议删除 1、保留 187、需人工判断 34。唯一一条建议删除的三个维度全部实测——去掉 windows.h 后编译 1610 ms → 910 ms（−43.5%）、GGC 内存 301,052 kB → 168,960 kB、目标文件 63,004 B → 62,972 B（−32 B，收益全部在编译时间侧）。187 个「保留」与 1 个「建议删除」同样是一等公民：把试过、不行记录下来，和把有效改动找出来一样重要。

3.5 四道门槛：我们把自己的卖点证伪了一次

在写这份文档之前，项目按预注册的判据跑了四道门槛，结果并不全是好消息。把它写在正文而不是附录里，是因为评审有权先看到不利数据。

G1 公平对照（部分通过）：与 ClangBuildAnalyzer 在同一批 25 个编译单元、同一编译器、同一候选规则下对照。把两边口径统一成「自身成本 × 次数」后，Top-10 重合 7/10、Spearman ρ ≈ 0.68——只能说明中度一致，不能说明我们更准。

G2 优化闭环（成立但有限）：第一版协议在 nlohmann/json 上跑出 0/10，当时据此判为不成立。把那个 0 拆开看，它有三个来源，逐个修掉之后结论变了。① 候选空间太窄——只看编译单元自己的 include，83 条候选里只有 3 条疑似未使用；把 include 图摊到所有文件后有 432 对，一条多余 include 的收益是「独占行数 × 受影响的编译单元数」。② 测量协议太钝——先测完基线再测补丁，机器漂移会整块算成收益，聚合 A/A 实测 2.005%，改成交替测量后降到 0.644%。③ 一处判定被自己污染——「#include <regex>」这一行的文件名会进入包含者的标识符集，用法判定于是恒判「在用」。修完之后：doctest（24 单元）上带可行性闸门的 16 条候选全部测完——8 条能编过、1 条被证实可删且真省时——删掉一个公共头里的 `#include <unordered_set>`，24 个单元全部编译通过，聚合耗时 20556 → 19937 ms（−3.01%）并通过复测；关闸门的 5 条里 0 条能编过。关掉闸门后，zlib（15 单元，C 语言）也找到一条：`zutil.h` 里删掉 `#include <stdlib.h>`，9 个单元全部通过，3314 → 3244 ms（−2.125%）并通过复测——这条正是闸门漏掉的，它被 `size` 这个词判成了「在用」。json（25 单元）仍是 0。三个工程的全量命中率是 1/16、1/16、0/16；放行的 16 条里还有 8 条编译不过——闸门只管筛，能不能编由 verify 说话。三档全量探测把闸门的作用量出来了：放行 16 条通过率 50%、加「无法判定」19 条 53%、完全不筛 66 条 47%——闸门对「能否编译」没有预测力，它省的是候选数量（verify 成本降到 1/4）。它也没漏掉收益：闸门没放行但能编过的 23 条里，初筛唯一超过 2% 的那条复测是 −0.548%。另做注入-召回：人工注入一条确实多余且昂贵的 include，头文件级排 1/17、单元级排 25/61，两条都被实测证实（−6.82%/−6.14% 与

−5.77%/−6.65%，各跑两遍独立运行都过线）。因此撤回「自动找到优化」的说法，但可以说在真实工程上产出过一条经复测的优化。闸门一条都不放行时，报告也不会只剩一张空表：它列出成本集中在哪（json 有 18.7% 的解析量落在自有头、zlib 是 41.1% 且前 10 个头占 73.4%）、收益最高的几条分别被什么挡住（精确到标识符，例如 zlib 的公共头在 15 个单元里用了 UINT_MAX），以及下一步该换什么手段（拆伞头、前向声明、PCH）。

G3 门禁（成立，但范围有限）：注入一次已知回归后 3/3 检出；无回归时 3/3 通过、零误报；构建损坏导致编译单元减少时 3/3 拦截（这一条是实测中发现缺陷后修的：旧逻辑会因为总量「下降」而放行）。已知盲区：行数不变但编译变慢的回归测不出来。

G4 工程开销（成立）：同任务下扫描开销为纯编译时间的 1.15×（GCC）/ 1.20×（Clang），峰值内存 9.2 MB，三次运行结果完全一致。

把四条放在一起，产品的诚实定位是：它不承诺替你找到优化，它承诺把「看起来能省」变成「确认能省」或「确认不能省」，并把这个结论固化成能拦住回归的基线——并且在 doctest 这个真实工程上，它确实产出过一条经复测的优化；但三个工程各只有 0 到 1 条，其中 zlib 那条还要关掉闸门才找得到，这些数字都写在上面。这个定位比原来的说法窄，但每一条都有可复现的实验数据支撑。

四、真实项目验证

第三章的数字来自刻意构造的工程，只能证明工具能跑。本章用真实开源项目检验它是否可信——尤其是它会不会误报。

4.1 在 fmt 上的扫描

对 4 个真实开源项目扫描。每个项目的 flags 都是实际跑通过、能产出干净数据的那一组，写在仓库 `benchmarks/run_all.py` 里而非让人记——复现不了的数字在评审眼里等于没测。

项目	编译单元	依赖边	头文件解析量	成本候选	疑似未使用
fmt	3 / 1 失败	651	331,016	4	0
nlohmann-json	25 / 0	9,294	4,866,676	24	0
spdlog	8 / 17 失败	3,246	1,579,902	41	0
cpp-httplib	18 / 7 失败	14,724	7,523,316	18	0
合计	54 成功	27,915	14,300,910	87	0

唯一失败的 `src/fmt.cc` 已确诊：该文件是 C++20 module（第 8 行 `module;`），MinGW GCC 16.2 不支持 `-fmodules`，换 `--std c++20` 同样报错。这是工具链限制，不是工具缺陷，且失败被如实报告而非静默跳过——报告顶部会写明「1 个编译单元失败，数字可能偏低」。

4.2 关键结果是那个 0

fmt 是维护良好的库，它的 include 确实是干净的。工具给出 4 个成本候选，对前 3 个逐一实测：

```
## windows.h ← os.cc
状态：compile-error — os.cc:90:9: error:
'FORMAT_MESSAGE_ALLOCATE_BUFFER' was not declared in this scope

## format-inl.h ← format.cc
状态：compile-error — format.cc:10:1: error:
'FMT_BEGIN_NAMESPACE' does not name a type

## os.h ← os.cc
状态：compile-error — os.cc:102:12: error:
'fmt' does not name a type
```

三个全部正确判定为「删了会编译失败」。一个只会喊「你可以删掉这个」的工具，第一次误报就会被人关掉。HeavyHeader 在成熟项目上报告「没有可删的」，同时仍然量化出成本集中在哪——这个对比本身就是价值。

把 4 个项目合起来看：87 个成本候选中 0 个疑似未使用。这个比例本身就是结论——成熟项目的 include 大多是干净的，工具没有为了凑数字而喊。反过来说，当它真的报出一个候选时（见第三章），那个候选值得认真对待。

与之对照，第三章构造的未使用 include 被准确识别并实测省 38.5%。该报的报，不该报的不报，两边都有数据。

4.3 这些数字的边界

如实说明：本章数字取自 GCC 16.2.0 (MinGW-w64) + Python 3.13，随编译器版本、平台、机器负载变化，不应逐位复现。可复现的是结论方向：候选数是否为 0、verify 是否报 compile-error。完整复现步骤见仓库 `benchmarks/results/README.md`。

4.4 完整构建对照：估算与实测分两张表

前面的扫描回答「成本在哪」，这一节回答「按它的结论动手，完整构建到底快了多少」。对象是 nlohmann/json v3.11.3 的测试目标（63 个编译单元，源码按 SHA-256 固定），按工具估算实施 PCH 后两臂各 5 次干净构建，63 个测试二进制逐个运行、逐套件比对断言数：

指标	基线	加 PCH	结果
完整构建墙上时间（中位数）	43.2 s	40.1 s	−7.1%
编译 CPU 合计（中位数）	292 s	259 s	−11.4%
63 个测试套件断言数	4,264,414	4,264,414	逐个一致

估算与实测分两张表，从不合并：工具估算「消掉 91.2% 的头文件解析量」，实测是「编译 CPU −11.4%」。反推下来头文件解析约占编译 CPU 的 17%——两个数不同，因为量的不是同一件事；

把估算当秒数引用是错的，本项目在任何文档里都不这么用。第一版实验里 PCH 曾让两个套件的断言数变化（配置宏在 include 之前被 `#define`，PCH 固化了宏状态）——门禁抓到后改成「凡在 include 之前改配置宏的单元一律不用 PCH」（16/63），覆盖率从 63 降到 47，收益也跟着降。这一步连同逐单元对照都写在实验报告里，不悄悄跳过。

4.5 落地到最后一公里：上游 PR 战役

实验证据落在克隆的副本上是不够的——这个场景的终点是真实工程真的接受这个改动。项目把两条已实测的优化做成了 PR 级补丁（仓库 `contrib/upstream/`），基于上游 HEAD 生成、对全新源码树 dry-run 验证可套用，并附英文 PR 正文（含实测数据与验证边界）：

工程	改动	实测依据
doctest	删 prelude.h 未用的 `#include <unordered_set>`	24 单元全部编译通过；聚合 20556 → 19937 ms（−3.01%），复测通过
zlib	删 zutil.h 未用的 `#include <stdlib.h>`	9 个受影响单元全部通过；3314 → 3244 ms（−2.125%），复测通过

每条 PR 正文都写明验证覆盖到哪里、没覆盖到哪里（下游使用者、其他平台工具链）。三个工程的可采纳率是 1/16、1/16、0/16——「自动帮你找到优化」不成立，「可验证地找到并证实一条、并把它变成上游改动」成立。结果无论合并与否，都会更新进仓库状态表与复盘文档。

五、现有工具覆盖不到的三件事

第一章已经交代清楚：成本排序、逐头文件耗时、结构建议、注释后重编，这四件事分别有成熟工具在做（Header Hero / ClangBuildAnalyzer / CompileScore / IncludeGuardian / deheader）。本章不重复它们，只讲三处现有工具覆盖不到的细节，以及每一处的证据边界——哪些是精确值，哪些只是启发式线索。

5.1 模板实例化成本归因

`-ftime-report` 从 GCC 9 起就报告 `template instantiation` 阶段，但对模板重型 C++ 它常是最大单项。需要说明的是：ClangBuildAnalyzer 已经在 Clang 上展示「最慢实例化的模板」（数据来自 `-ftime-trace`）。我们补的是另外两件事：把同一份归因带到 GCC 与 MSVC（靠 `-ftime-report`），以及把成本按头文件摆出来，而不只是按模板名列一个全局 Top-N。

实测：一个包含 `nlohmann/json.hpp` 的文件，模板实例化 `0.44s` / 总耗时 `1.13s`，即 39%。在 nlohmann-json 的 25 个编译单元上，全项目模板实例化占 42.8%，22 个单元占比超过 25%。

> 模板成本分两部分，诚实程度不同，报告里分别标注：

指标	来源	性质
每个 TU 的秒数与占比	-ftime-report	精确
每个头文件的「模板表面」	文本启发式（template< 声明数 × TU 数）	线索，非结论

按头文件的归因编译器给不了——它只报告阶段总量。所以「模板表面」只能提示成本可能来自哪里。在 nlohmann-json 上，头号嫌疑是 `json.hpp`（173 个模板声明 × 24 个 TU）。系统头文件（如 `<type_traits>`）的模板声明数通常更高，但用户改不了，故不列入报告。

5.2 预编译头（PCH）收益估算

PCH 是慢构建里杠杆最大的单项改动，但多数团队靠猜决定放什么——通常是「大家都包含的头文件」，这经常是错的。本工具从 include 图算出该放什么。

```
候选 PCH 的头文件集合 P:
收益 = Σ lines(h), h ∈ closure(P) × 已包含全部 P 的 TU 数
− Σ lines(h) （一次性构建成本）

// nlohmann-json 实测
建议纳入 5 个头文件，传递闭包 139,180 行，
24/25 个编译单元已全部包含它们（覆盖率 96%），
估算净收益 3,201,140 行（占全项目头文件解析量 65.8%）。
```

四条限制写进报告本身：行数是代理不是秒数；上限估计（真实 PCH 必须是每个 TU 的第一个 include 且字节一致）；未计 PCH 失效；本工具不生成 PCH。

5.3 前向声明建议

把 include 换成前向声明，是编译时间优化里收益最高的一类：编译器读一行文本，而不是整个头文件的传递闭包。也是最容易出误报的一类，因为「只以指针/引用形式使用」是语义属性，而我们在文本上判断。

所以规则刻意保守，且报告给出具体类型名、使用行号、阻断类型：

```
| 编译单元 | include | 可前向声明的类型 | 使用行号 | 阻断类型 |
| example\hello.cc | httplib.h | Request | 14 | Response、Server |
```

「阻断类型」非空表示该头文件里还有其他类型需要完整定义，不能整个 include 换掉，只能把可前向声明的类型拆出去。开发过程中修掉一个真实的假阴性：`int read(Value* v) { return v->n; }` 里 `Value` 只以指针位置出现，纯文本检查会判为可前向声明，但 `v->n` 需要完整类型。补上「跟踪指针变量是否被解引用」后正确判为阻断。

六、平台能力：从工具到流水线的一环

如果只能跑一次、出一份报告，它是个玩具。HeavyHeader 的九个命令覆盖「发现 → 验证 → 门禁 → 对比 → 破案 → 看趋势 → 进 PR」的完整闭环：

命令	作用	在流水线中的位置
`scan`	成本归因，产出 Markdown / HTML / JSON 报告，可存基线	本地分析、CI 报告
`verify`	注释掉候选 include、重编译、实测耗时/内存/体积差异	采纳建议前的确认
`gate`	成本相对基线回归超阈值则非零退出	PR 门禁
`compare`	对比两份扫描，定位成本上升的具体头文件	版本间对比
`blame`	把成本回归归因到 commit、文件与具体的 include 行	门禁报警后的破案
`edge-cost`	按 include 边归因解析成本（clang-first，需真编译一次）	候选排序与热点定位
`toolchains`	同一份代码在多个编译器下的成本横评	工具链迁移决策
`history`	多次扫描聚合成时间序列，终端摘要 / 自包含网页 / 本地看板	成本趋势治理
`pr-comment`	把两次扫描对比渲染成 GitHub 风格评论，可选经 gh 幂等发布	PR 自动评论

6.1 CI 门禁为什么可信

门禁只对比确定性的归因行数和相对比例，不对比墙钟时间——后者随负载波动，拿它做判定会产生误触发。而一个误触发的门禁会被关掉，关掉的门禁保护不了任何东西。默认阈值 5% 是「明显变化」与「正常波动」之间的实用分界。

失败的构建会指名道姓：

```
门禁失败：
- header cost regressed +17.1% (206100 -> 241300 attributed lines),
budget is +5.0%
worst offender: include/bloated.h 0 -> 22312 lines (+100.0%)
```

一个只说「成本回归 +12%」的门禁毫无用处；能定位到具体文件和具体 include 行的门禁才会被留下来。

6.2 报告是自包含单文件

HTML 报告不引 CDN、不引框架、图表是手写内联 SVG。这不是技术偏好，是需求：报告要贴进 PR、要发给别人、要在没网的机器上打开。一个在这些场景下显示白页的报告毫无价值。数据量级（几十个头文件、几十个历史点）也远不够撑起一个图表库的开销。

6.3 可部署性：零依赖是一条被测试守护的约束

运行时零第三方依赖（Python 3.10+ 标准库）不是口号，是 CI 里的两条断言：`pyproject.toml` 不得声明运行时依赖、`src/` 只许导入标准库——未来任何新增依赖都会让 CI 直接失败（`tests/test_no_deps.py`）。这让工具能在最朴素的 CI 容器里一条命令装好、跑起来。交付形态：源码安装 + 打 tag 即由 `release.yml` 自动构建并校验「装得上、CLI 可用」的发布件。

6.4 成本考古：门禁报警之后，谁干的

门禁说「成本回归 +17.1%，主要是 include/bloated.h」，然后呢？在几百个提交里找是谁把这条 include 加进来的，只能靠人翻——这一步不做，报警就只是一张罚单。`blame` 把这一步变成一条命令：它读两份基线（扫描时自动记录了当时的 git commit），用 `git diff` 解析出两次之间的 include 行变化，再按「可达目标且新增/失去它的编译单元集合完全一致」确定影响半径。

```
$ heavyheader blame base.json pr.json --repo .
成本 +56,487 行（206,106 → 262,593）
commit d37bcca4 c3: core.h pulls bloated.h
file include/core.h
change +#include "bloated.h"
预测 +56,485 实测 +56,485 置信度 high
未解释 +2 行（被改文件自身的行数变化）
```

预测与实测分行列出、不相加：预测是「这条边影响的头文件行数 × 受影响的 TU 数」，实测是两份基线的差，两者对得上才是结论，对不上就进「未解释」——上面那 +2 行正是被修改文件自己的行数变化，工具不把它硬塞进任何一条 include 的账上。

这条能力做过全网搜查（GitHub 仓库检索 + 项目官网 + 学术检索，2026-10-19）：相邻的 `build-time-blame-plugin` 分析的是 Jenkins 控制台历史、`build-blame` 剖析单次构建、Bencher 一类只画耗时趋势——把成本增量归因到具体 commit 与 include 行的工具，搜查未见同能力实现。

6.5 边级归因：从「哪个头贵」到「哪一行贵」

`scan` 回答「哪个头文件贵」（节点级），但开发者能删的是源文件里的那一行（边级）。两者不是同一个量：一个头被五个编译单元共享时，删掉其中一条 include 只省下这条边独占的那部分。`edge-cost` 就答这一句：一次真编译同时开 `-ftime-trace` 与 `-H`，trace 的嵌套区间还原逐文件自耗时，每条边按「独占闭包」求和，再跨编译单元聚合。

这里有一个不做就会系统性高估的坑：`-H` 只记录每个头首次生效的包含点，第二次包含被 include guard 吞掉、根本不出现在树里。于是「删掉 prelude.h 里的 <cmath>」在树上看起来只通向 cmath，实测删掉之后该编译单元的解析文件数一个不少——cmath 早被别的头拉进来了。修法是把树内每个文件的 `#include` 行文本扫描一遍，补回被抑制的替代供给点；用「删掉后真消失的文件集合」作确定性真值验证：25 条可删边的假阳性由 45 个文件降到 0，完全一致率 18/25 → 24/25。

七、技术路线与架构

7.1 分层

```
src/heavyheader/
├── drivers/ 向编译器提问：GCC/Clang/MSVC 各自的方言
├── parse/ 解析 -H 依赖树、-ftime-report 计时、make 依赖文件
├── attrib/ 纯函数：依赖图、独占闭包、重复解析、使用启发式
├── verify.py 实测：注释掉 include → 重编译 → 计时
├── scanner.py 编排：发现 TU → 线程池并行编译 → 汇总
├── baseline.py 基线、对比、CI 门禁
└── report/ Markdown / HTML / JSON 渲染
```

关键性质：`attrib/` 和 `parse/` 是纯函数，不碰编译器、不碰文件系统、不碰网络。核心逻辑可以被彻底单元测试——归因引擎的全部测试用例都不需要安装任何编译器。

7.2 为什么核心用 Python 而不是 C++

项目的使用者是 C++ 开发者，但工具本身用 Python 写。理由是工作性质：它的主体是调用子进程、解析文本、建图、排序、渲染报告——这是 I/O 密集型而非计算密集型，Python 的开发速度优势直接转化为交付速度，且零第三方运行时依赖意味着它能在最朴素的 CI 容器里跑起来。编译慢的 C++ 项目不该为了分析工具再拖一套运行时进去。

需要声明的是：这个选择有代价。Python 的解释器启动开销在每次调用编译器时都会被放大，而 libclang 绑定在 Python 生态里更成熟。Roadmap 中的「精确模式」会利用后一点。

7.3 支持的工具链

编译器	依赖树	分阶段计时
GCC 9+	`-H`	`-ftime-report`
Clang 17+	`-H`	`-ftime-report`
MSVC	`/showIncludes`	墙钟计时（无等价文本格式）

有 `compile_commands.json` 时严格按其中的 flags 编译——同一份代码在不同 flags 下的数字没有可比性。

八、诚实的边界：不做什么

这一章和前面几章同样重要。一个工具如果把局限藏起来，使用者就无法判断什么时候该信它、什么时候不该。

不做什么	为什么	怎么办
不判定 include 是否「未使用」的语义真相	文本启发式只能回答「源码里有没有出现该头文件声明的标识符」	权威结论由 verify 给出：真的删掉、重新编译
不做精确的 AST 级分析	要精确判定需要 libclang，会把使用门槛抬到需装整套 LLVM 工具链	Roadmap v0.3 作为可选 extra，默认路径保持零依赖
不改写代码	自动重构一个它不完全理解的代码库，风险远大于收益	工具给建议，人做决定；verify 只在临时目录做实验
不跨机器比较绝对耗时	墙钟时间随负载波动	门禁只对比确定性的归因行数与相对比例
不信任编译器的自报时序	实测发现 GCC on MinGW 会把某阶段报成 584 年（见 8.1）	用进程墙钟做上界校验，不可能的的行丢弃并警告

8.1 一个真实的编译器缺陷

开发过程中在 fmt 上发现 `baseline_seconds` 报出约 1170 年。根因不在本项目：GCC 16.2.0 (MinGW-w64) 的 `-ftime-report` 在编译包含 `<windows.h>` 的文件时，会把某个阶段报成 `18446744070.35` 秒——即 2^64 纳秒折成秒，约 584 年；总数是若干这样的阶段累加出来的。

解析器忠实解析了垃圾，于是报告里出现一个看起来权威实际荒谬的数字。修法：driver 本来就用 `perf_counter` 测量了进程墙钟时间，而一个编译阶段不可能比包含它的进程更长。以此为上界校验，超出的行丢弃，并 surfaced 成警告而非静默修正——用户有权知道他们的工具链有缺陷。

这件事本身说明了「不自己解析 C++」这条原则的代价：你把编译器当权威，就得防着它说谎。所以工具对编译器的每个输出都保留交叉校验的能力。

8.2 已知局限汇总

① 行数不等于解析难度，重型模板头的权重被低估；② 默认 `-fsyntax-only`，不含代码生成阶段成本；③ 使用状态是文本启发式，为副作用而包含的头文件（如 `<iostream>` 的流初始化）会被误判；④ 增量编译（ccache、distcc）会改变耗时分布，跑 verify 前需清缓存。完整列表见仓库 `docs/methodology.md` 第 9 节。

8.3 一次被我们自己否决的精度声明

6.5 说边级归因能给出「每条 include 删掉后省多少毫秒」。那么它比行数排序更准吗？这个问题只有实测能答，我们做了：doctest 的 24 个编译单元、prelude.h 里 36 条候选，逐条真删真编，6 个单元 × 3 轮顺序交替（基线/改动、改动/基线…，抵消「先跑的总是吃冷缓存」这个系统偏差），取中位数。整轮 19 分钟。

结果不好看：25 条可删边的实测节省分布在 −168 ms 到 +305 ms 之间，符号都不稳定；毫秒排序、行数排序与实测节省的秩相关分别是 −0.21 与 −0.19，都是负的。原因不复杂：基线单编译单元 0.67 s，单条标准头只解析几毫秒——低于这台机器的分辨率。

所以本项目不声称「毫秒排序比行数排序更准」。这句话缺证据，缺证据就不写进任何材料——包括这份文档。边级毫秒只用于排序候选，一条 include 值不值得删，仍然只能由 verify 的 A/B 实测回答。

但同一批实验抓到了一个真 bug，并且修好了：`-H` 只记录每个头首次生效的包含点，被 guard 吞掉的替代路径看不见，于是「独占闭包」系统性高估收益——25 条可删边合计虚报 45 个「会消失但其实不会消失」的文件，20 条出边里 7 条承诺了根本不存在的收益。改用「删掉后真消失的文件集合」这个确定性真值（不受计时噪声影响）验证：补全潜在供给边后，假阳性 45 → 0，完全一致率 18/25 → 24/25。

这件事的方法学结论比功能本身更重要：细粒度预测不能用墙钟验证，噪声会淹没信号；要用「哪些文件还在被解析」这种确定性集合去验。实验脚本、原始数据与全部负面结果在仓库 `experiments/edge-precision-20261019.md`。

九、开源治理与社区

项目从第一行代码起就按可协作的标准组织，而不是跑出 Demo 再补文件。大赛评分中「开源治理」占 20%，但我们做这些的理由更直接：一个没有 CONTRIBUTING 的项目收不到贡献，没有行为准则的社区留不住人。

文件	作用
README.md	问题定义、与同类工具对比、快速开始、诚实的边界
LICENSE (Apache-2.0)	对贡献者和使用者都明确，含专利授权条款
CONTRIBUTING.md	按优先级列出「什么贡献最被需要」，避免力气花错方向

文件	作用
CODE_OF_CONDUCT.md	基于 Contributor Covenant 1.4
SECURITY.md	威胁模型：编译器输出是不可信输入、临时文件、不联网
GOVERNANCE.md	治理结构：角色与贡献者阶梯、决策流程、发布纪律、依赖政策
ROADMAP.md	v0.1–v0.5 路线图 + 「明确不做的事」清单
CHANGELOG.md	语义化版本，记录每个修复的根因
docs/innovation.md	独创性说明：CBE 范式、三项创新点、消融表、不声称清单
docs/scenario.md	场景落地：四步全链路 + 真实工程数据卡 + 复现入口
docs/methodology.md	每个指标的定义、局限、以及「查不到就不该出现在报告里」
docs/architecture.md	分层设计与刻意选择的理由
tests/test_no_deps.py	CI 守护零运行时依赖（架构约束变成会失败的测试）
.github/	Issue 模板（bug/feature）、PR 模板、CODEOWNERS、4 个工作流（CI / 覆盖率 / MSVC 证据 / 发布）

9.1 CI：16 个岗位，当前全部通过

CI 覆盖 Linux / Windows / macOS 三个操作系统 × Python 3.10–3.13 四个版本的测试矩阵（12 个岗位），外加 lint（ruff）、自举扫描、覆盖率与 MSVC 证据，共 16 个岗位（3 个工作流：CI 14、覆盖率 1、MSVC 证据 1）。self-scan 任务在 CI 里构造一个真实 C++ 工程并用 HeavyHeader 扫描它，断言归因行数非零、编译单元被发现——这一步同时也是集成测试，验证工具在非 Windows、非 MinGW 的工具链下也能正确解析编译器输出；MSVC 证据任务在 Windows runner 上真实运行 `cl` 并跑门禁。

2026-10-19 修复了一个此前无人发现的慢性问题：仓库的 CI 自建立起每个提交都是红的——本地只有 Windows，7 个跨平台测试与 3 个 semantic 测试在 Linux/macOS 上失败（Windows 路径解析的平台依赖），全部修复后于当日转为全绿。这件事本身说明了矩阵的价值：只在一种操作系统上绿的测试套件，不等于绿的测试套件。根因分析与修复记录在 CHANGELOG [未发布]。

9.2 为什么选 Apache-2.0

三个理由：对贡献者和使用者都明确；含专利授权条款，这对一个要进企业 CI 的工具不是形式问题；与生态中大量 C++ 项目的许可证兼容。

9.3 测试策略

544 个单元测试（另有 2 个需要 clang++ 的用例在无 clang 的机器上跳过）。样本优先用真实编译器输出逐字复制——例如 `tests/test_parsers.py` 里的 `-H` 和 `-ftime-report` 输出是从 GCC 16.2 真实运行复制的。这意味着换编译器版本时这些测试会最先报警。测试按平台中性编写：Windows 风格路径的期望值一律用规范形式（正斜杠），不写只在一端成立的表达式。

十、开发过程与后续计划

10.1 已修的真实缺陷

开发过程中修掉多个 bug，每个都有回归测试。列出它们是因为它们解释了这个项目为什么是现在这样：

缺陷	症状	为什么难发现
独占闭包恒为空	所有项目都报「无可优化项」	症状与「确实没问题」在输出上完全无法区分
make 规则按第一个裸冒号切分	Windows 盘符冒号被误判为分隔符，目标被截断	只在 Windows 路径上出现，Linux 开发不会遇到
MSVC 缩进深度直接除步长	所有嵌套 include 都报成深度 0	MSVC 最浅层本身有空格，不是从 0 开始
usage_label 漏序列化	基线文件丢失使用状态信息	模型加了字段但序列化器没写，两边不同步
GCC 坏计时器（见 8.1）	构建成本报出 1170 年	编译器自己的输出看起来是权威的
源文件被传了两次	compile_commands 模式下所有计时建立在一遍编译上	GCC 安静地编译两遍，读取方只取第一条 TOTAL
SemanticVerdict 从未导入	models.py 引用了一个不存在的名字	靠 from __future__ import annotations 的惰性注解隐身，一调 get_type_hints 就炸
Windows 路径解析的平台依赖	本地 Windows 全绿，CI 12 矩阵全红（每个提交都红）	os.path.normcase 在 POSIX 是 no-op；PureWindowsPath 不经过 as_posix() 时渲染行为跨 Python 版本不一致
两个测试整段重复定义	前两份同名测试从未被执行	复制粘贴后第二次定义静默覆盖第一次，无任何报警
边级闭包按「首次包含」计算	25 条可删边虚报 45 个「会消失但其实不会消失」的文件（见 8.3）	-H 只记首次生效的包含点，被 include guard 吞掉的替代路径看不见；症状是「预测省了很多、实测一分没省」

缺陷	症状	为什么难发现
编译命令缺 -c（edge-cost 兜底路径）	clang 一路链接下去，Windows 上报 lld-link: undefined symbol	报错长得像源码有错，其实是工具在让它链接

10.2 Roadmap

版本	内容	状态
v0.1	可用的核心：三种工具链解析、成本归因、四个命令	已交付
v0.2	从工具到平台：趋势看板、增量扫描、PR 评论机器人、多编译器横评	已交付
v0.3	精确模式：独占闭包判定、可选 AST 交叉检查、前向声明建议、PCH 收益评估	已交付
v0.4	精度引擎：成本考古（blame）、边级归因（edge-cost）	已交付
v0.5	生态集成：CMake、Bazel/Meson、VSCode 插件、IWYU 输出互转、PyPI 上架	计划中

Roadmap 里还有一份「明确不做的事」清单：不做通用静态分析器、不自动修改代码、不做云服务、不追求支持所有编译器。这份清单和功能列表同样重要——它防止项目范围蔓延。

10.3 关于团队

本项目由个人开发者以「开源社区」主体类型申报，独立完成需求、设计、实现、测试与文档。这是一个明确的事实，不是需要修饰的短板：项目从零到可运行的全部工作都在公开仓库的 git 历史里，每个 commit 都说明了「为什么」而不只是「做了什么」。

后续开发按 Roadmap 推进，欢迎社区参与。`CONTRIBUTING.md` 开头按优先级列出了「什么贡献最被需要」——排第一的是真实项目的扫描结果反馈，因为那是唯一能检验工具在真实代码库上是否给出荒谬结论的方式。

十一、参考资料

11.1 实际核读过的同类项目

以下项目在 2026 年 10 月 18—19 日两轮逐项核读（能力、限制、平台均以官方README / 文档 / 发布页为准），用于第一章的竞争格局表。逐格证据等级与复核清单见 `docs/competitor-matrix.md` 与 `docs/competitors.md`。

```
R1 Header Hero (Bitsquid, 2011)
bitsquid.blogspot.com/2011/10/caring-by-sharing-header-hero.html
fork: github.com/aras-p/header_hero
R2 ClangBuildAnalyzer (aras-p)
github.com/aras-p/ClangBuildAnalyzer ; aras-p.info/blog/2019/09/28/
```

R3 CompileScore（Viladoman, MIT）
github.com/Viladoman/CompileScore（VS 扩展 + 独立应用 + Data Extractor）
R4 IncludeGuardian（v0.0.8）
includeguardian.io ； includeguardian.io/instructions
R5 deheader（Eric S. Raymond, v1.13）
catb.org/~esr/deheader/ ； catb.org/~esr/deheader/deheader.html
R6 include-what-you-use / clang-include-cleaner
include-what-you-use.org ； github.com/include-what-you-use/include-what-you-use
R7 微软 C++ Build Insights
learn.microsoft.com/cpp/build-insights/get-started-with-cpp-build-insights
R8 Include-Wrangler（2012）
github.com/lukedodd/include-wrangler
R9 clang-build-profiler（MIT）
github.com/DmitriBogdanov/clang-build-profiler
dmitribogdanov.github.io/clang-build-profiler/
R10 include-what-costs（BSD-3, LHCb）
github.com/chrisburr/include-what-costs ； pypi.org/project/include-what-costs/
R11 基准对象：nlohmann/json（github.com/nlohmann/json）、fmt（github.com/fmtlib/fmt）、spdlog、cpp-httplib、doctest、zlib（madler/zlib）

11.2 实际使用的编译器文档

R12 GCC 手册：-H、-ftime-report、-M/-MM 选项说明
R13 MSVC 文档：/showIncludes 编译器选项
R14 C++ 标准：include guard 与 #pragma once 的语义

11.3 未引用但想说明的事

本项目没有引用政策文件或学术论文来支撑其必要性。这不是疏漏，是刻意的：我们没有找到一份能直接支撑「头文件成本归因」这个具体问题的权威文献，而为了显得严谨去引一份关系不大的政策条文，比不引更糟。

项目的必要性论证来自三件可验证的事：第一章的成本模型（可手算）、第三章的构造工程（可复现）、第四章的真实项目数据（可复现）。如果这三件事不成立，引多少文献也救不了它。

本作品以 Apache-2.0 许可证开源提交。代码仓库、测试、治理文件与基准数据均已随作品材料提交。

> 代码仓库：https://github.com/wzzrambler/heavyheader
> 演示视频：随作品材料提交（3-5 分钟完整片）
> 复现方式：见仓库 benchmarks/README.md，本文所有实测数字均可一键重跑。